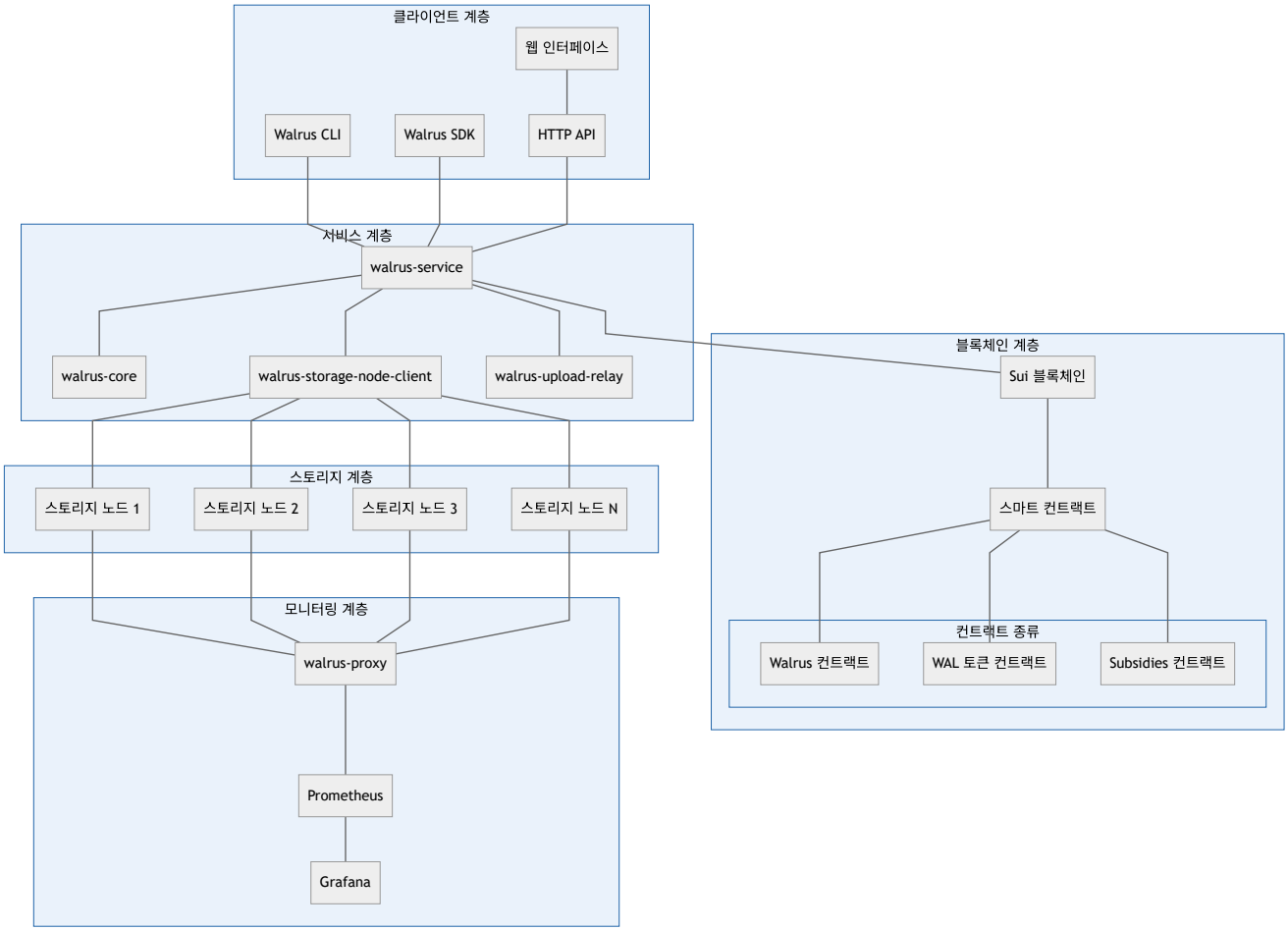
# Walrus 시스템 개요

## 소개

Walrus는 Sui 블록체인을 기반으로 한 탈중앙화 블롭 스토리지 시스템입니다. Reed-Solomon 인코딩을 사용하여 데이터를 여러 스토리지 노드에 분산 저장하고, 비잔틴 장애 허용(Byzantine Fault Tolerance)을 통해 높은 가용성과 내구성을 제공합니다.

## 전체 시스템 아키텍처


클라이언트 계층
웹 인터페이스
Walrus CLI
Walrus SDK
HTTP API
서비스 계층
walrus-service
walrus-core
walrus-storage-node-client
walrus-upload-relay
블록체인 계층
Sui 블록체인
스마트 컨트랙트
컨트랙트 종류
Walrus 컨트랙트
WAL 토큰 컨트랙트
Subsidies 컨트랙트
스토리지 계층
스토리지 노드 1
스토리지 노드 2
스토리지 노드 3
스토리지 노드 N
모니터링 계층
walrus-proxy
Prometheus
Grafana

# 주요 컴포넌트

## 1. 클라이언트 계층

### Walrus CLI

- **역할**: 명령줄 인터페이스를 통한 블롭 저장/검색
- **주요 기능**:
- 파일 업로드 및 다운로드
- 블롭 메타데이터 조회
- 네트워크 상태 확인
- **바이너리**: `walrus`

### Walrus SDK

- **역할**: 개발자용 라이브러리 및 API
- **주요 기능**:
- 프로그래밍 방식 블롭 조작
- 인증 및 권한 관리
- 비동기 작업 지원
- **크레이트**: `walrus-sdk`

## 2. 서비스 계층

### walrus-service

- **역할**: 핵심 서비스 로직 및 바이너리 구현
- **주요 바이너리**:
- `walrus` : 클라이언트 바이너리

- `walrus-node` : 스토리지 노드 바이너리
- `walrus-deploy` : 배포 도구
- `walrus-backup` : 백업 도구

**walrus-core**

- **역할**: 핵심 타입 및 인코딩/디코딩 로직
- **주요 기능**:
- RedStuff 인코딩 알고리즘
- 머클 트리 구현
- 비잔틴 장애 허용 로직
- 기본 데이터 구조 정의

**walrus-storage-node-client**

- **역할**: 스토리지 노드와의 HTTP 통신
- **주요 기능**:
- RESTful API 클라이언트
- TLS/HTTPS 보안 통신
- 연결 풀링 및 재시도 로직

## 3. 블록체인 계층

**Sui 블록체인 통합**

- **역할**: 메타데이터 저장 및 거버넌스
- **주요 기능**:
- 블롭 메타데이터 등록

- 스토리지 노드 관리
- 결제 및 인센티브 시스템

**스마트 컨트랙트**

- **Walrus 컨트랙트**: 핵심 스토리지 로직
- **WAL 토큰 컨트랙트**: 네이티브 토큰 관리
- **Subsidies 컨트랙트**: 스토리지 보조금 시스템

### 4. 스토리지 계층

**스토리지 노드**

- **역할**: 실제 데이터 저장 및 제공
- **주요 기능**:
- 슬리버(Sliver) 저장
- HTTP API 제공
- 메트릭 수집 및 보고
- **바이너리**: `walrus-node`

## 핵심 개념

### 블롭(Blob)

- **정의**: 저장되는 원본 데이터 단위
- **특징**:
- 최대 크기 제한 있음
- 고유한 블롭 ID로 식별

- 불변성 보장

### 슬리버(Sliver)

- **정의**: RedStuff 인코딩을 통해 생성된 데이터 조각
- **종류**:
- Primary Sliver: 원본 데이터 기반
- Secondary Sliver: 복구용 패리티 데이터
- **특징**: 각 스토리지 노드에 분산 저장

### RedStuff 인코딩

- **정의**: Reed-Solomon 기반의 삭제 코딩(Erasure Coding) 알고리즘
- **특징**:
- 비잔틴 장애 허용
- 효율적인 복구 메커니즘
- SIMD 최적화 지원

### 에포크(Epoch)

- **정의**: 스토리지 기간의 단위
- **특징**:
- 블록체인에서 관리
- 스테이킹 및 보상 주기
- 노드 재구성 시점

## 데이터 흐름 개요

### 저장 프로세스

1. **클라이언트 요청**: 사용자가 블롭 저장 요청
2. **인코딩**: RedStuff 알고리즘으로 슬리버 생성
3. **메타데이터 등록**: Sui 블록체인에 블롭 정보 기록
4. **슬리버 배포**: 각 스토리지 노드에 슬리버 전송
5. **확인**: 저장 완료 후 블롭 ID 반환

### 검색 프로세스

1. **클라이언트 요청**: 블롭 ID로 데이터 요청
2. **메타데이터 조회**: 블록체인에서 블롭 정보 확인
3. **슬리버 수집**: 필요한 수만큼 슬리버 다운로드
4. **디코딩**: RedStuff 알고리즘으로 원본 데이터 복원
5. **반환**: 복원된 데이터를 클라이언트에 전달

## 보안 모델

### 암호화

- **전송 중**: TLS/HTTPS를 통한 암호화
- **저장 시**: 선택적 클라이언트 측 암호화
- **인증**: 디지털 서명 기반 무결성 검증

### 접근 제어

- **인증**: JWT 토큰 기반
- **권한**: 역할 기반 접근 제어

- **감사**: 모든 작업 로깅

### 비잔틴 장애 허용

- **가정**: 전체 노드의 1/3 미만이 악의적
- **메커니즘**: 다수결 기반 합의
- **복구**: 자동 슬리버 재생성

## 성능 특성

### 처리량

- **저장**: 병렬 인코딩 및 업로드
- **검색**: 병렬 다운로드 및 디코딩
- **최적화**: SIMD 벡터 연산 활용

### 확장성

- **수평 확장**: 스토리지 노드 추가
- **부하 분산**: 자동 샤딩
- **캐싱**: 다계층 캐시 시스템

### 가용성

- **목표**: 99.9% 이상
- **복구 시간**: 분 단위
- **데이터 내구성**: 99.999999999% (11 9's)

## 다음 단계

이 개요를 바탕으로 다음 문서들을 참조하여 더 자세한 내용을 학습할 수 있습니다:

- 데이터 흐름 상세 분석
- 컴포넌트 관계도
- walrus-core 크레이트 분석
- 데이터 저장 프로세스

---

*관련 문서: 데이터 흐름 | 컴포넌트 관계도 | 용어집*